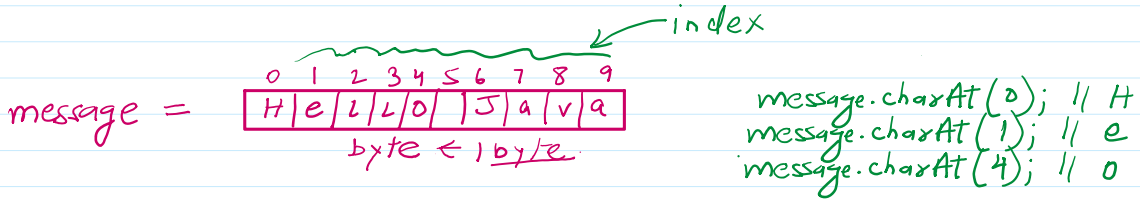
Java String Method Or String Class methods in Java

10 January 2026 09:43

In Java, the String class represents a sequence of characters. It is a fundamental part of the language that is used extensively for storing and manipulating. Text data. It has many built in methods that help us. To do string manipulation like slicing parts of a sentence, joining two pieces of text, changing the format, checking if two texts are same or replacing one word with another.

Java String Class Methods

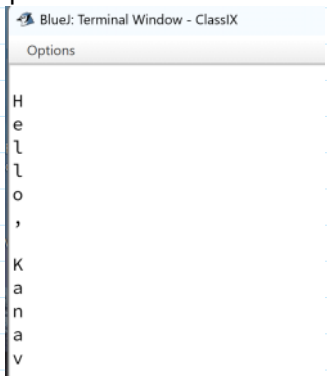
The java.lang.String class provides many useful methods to perform operations on sequence of values.



char charAt(int index)	It returns care value for a particular index.

```
public class DemoOf_charAt
{
    /**
     * Enter any string in args[]:
     */
    public static void main(String[] args){
        String h = args[0];
        int i;
        System.out.print("\f");
        for(i = 0; i < h.length(); i++){
            System.out.print("\n" + h.charAt(i));
        }
    }
}
```

Output:



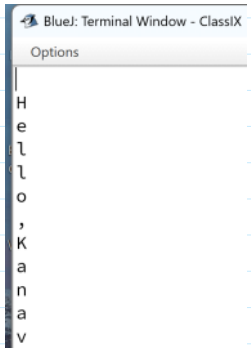
Tested for space:

```
package StringClasses;
```

```
/**
 * Write a description of class DemoOf_charAt here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class DemoOf_charAt
{
    /**
     * Enter any string in args[]:
     */
    public static void main(String[] args){
        String h = args[0];
        int i;
        System.out.print("\f");
        for(i = 0; i < h.length(); i++){
            char c = h.charAt(i);
            if(c != ' '){ //checking for space
                System.out.print("\n" + c);
            }
        }
    }
}
```

returns the string length.

Output:



WAP to input a string and calculate how many vowels, consonants, space and digits in it. For example:
Input: On 26th January, India is a republic country.
Output:
Number of vowels: 14
Number of consonants: 19
Number of digits: 2
Number of spaces: 7

String concatenation

String concat(String s)	It concatenates the specified String, we can use '+' operator to do the same.

```
String fname = "Kanav";
String lname = "Pasari";
String space = " ";
String name = fname.concat(space).concat(lname);
name
"Kanav Pasari"   (String)
```

```
String nm = fname + space + lname;
nm
"Kanav Pasari"   (String)
```

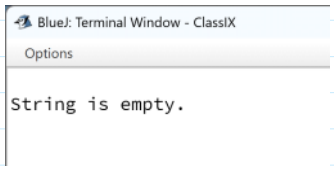
To check String variable or Object is empty or not

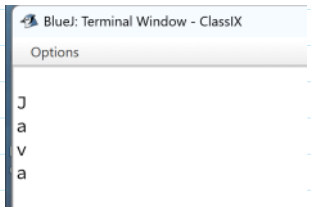
boolean isEmpty()	It checks if string is empty.

```
package StringClasses;

/**
 * Write a description of class DemoOf_charAt here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class DemoOf_charAt
{
    /**
     * Enter any string in args[]:
     */
    public static void main(String[] args){
        String h = args[0];
        int i;
        System.out.print("\f");
        if(h.isEmpty()){
            System.out.print("\nString is empty.");
        }
        else{
            for(i = 0; i < h.length(); i++){
                char c = h.charAt(i);
                if(c != ' '){
                    System.out.print("\n" + c);
                }
            }
        }
    }
}
```

Output:





← when string is not empty.

Homework

1. The "Custom Joiner" Logic

Given an array of Strings, write a method that concatenates them all into a single String, but places a specific "delimiter" (like a comma or hyphen) between each word. Constraint: Ensure there is no trailing delimiter at the very end of the final String.

2. Palindrome Validator
Write a program that takes a String and determines if it is a palindrome (reads the same forward and backward) using only the charAt() and length() methods. Ignore case sensitivity (e.g., "Level" should be true).

3. Vowel vs. Consonant Counter
Expand on the character-counting logic in the image. Write a program that iterates through a String and prints the total count of vowels, the total count of consonants, and the total count of special characters/spaces.

4. String Reversal without reverse()
Java's StringBuilder has a built-in reverse function, but for this challenge, use a for loop and the charAt() method to manually build a new String that is the reverse of the input.

5. First Non-Repeated Character
Write a program to find the first character in a String that does not repeat anywhere else in that String. If all characters repeat, return a message saying so.
Hint: You will need nested loops or a frequency array.

6. The "Empty or Blank" Sanitizer
In the image, you see isEmpty(). However, a String could contain only spaces " ". Write a method that checks if a String is either truly empty OR consists only of whitespace characters, without using the built-in .trim() or .isBlank() methods.

7. Substring Searcher
Without using the built-in .contains() method, write a program that takes two Strings: a source and a target. Determine if the target exists inside the source by comparing characters index by index.

8. Capitalization Formatter
Create a program that takes a sentence in all lowercase (e.g., "java is fun to learn") and converts it to "Title Case," where the first letter of every word is capitalized ("Java Is Fun To Learn").

9. Word Frequency Map
Given a long String of text, count how many times each specific word appears. You can assume words are separated by single spaces. Output the result in a list format.

10. String Compression (Basic)
Implement a method to perform basic string compression using the counts of repeated characters. For example, the string aabccccccaaa would become a2b1c5a3. If the "compressed" string is not smaller than the original string, your method should return the original string.